

Software Test Plan (STP) - Blood Bank Management System

Project: Blood Bank Management System (BBMS)

Version: 1.0

Authors: Team 9

Technology Stack: Python (Django) / JavaScript (Node.js), PostgreSQL/MySQL

Course: Software Engineering — UE23CS341A

Status: Approved for Execution

1. Introduction

Purpose: This document defines the test plan for the Blood Bank Management System (BBMS) v2.0. It outlines objectives, scope, strategy, resources, schedule, responsibilities, risks, deliverables, and release criteria.

Scope: Testing covers registration/authentication, donor management, blood inventory, blood requests and availability, donation camps, notifications, reporting, search/matching, administration, certificates, performance, safety, security, reliability, usability and data integrity.

References: IEEE 829-2008; ISO/IEC/IEEE 29119; BBMS System Design Document and API contract (REST/JSON); Team 9 coding and Git branching/issue-tracking conventions.

Definitions: BBMS, FR (Functional Requirement), NFR (Non-Functional Requirement), RBAC (Role-Based Access Control), OTP (One-Time Password), RTM (Requirements Traceability Matrix), STP (Software Test Plan), RACI (Responsible, Accountable, Consulted, Informed).

2. Test Items

- User registration, login, OTP verification and profile management
- Donor eligibility and donation history
- Blood inventory management and stock visibility
- Blood-unit expiry, traceability and disposal
- Blood request submission, validation, compatibility and reservation
- Donation camp/event management
- SMS/email notifications and emergency broadcasts
- Reporting and administrative dashboards
- Donor and blood-bank search/matching
- Administrator functions and audit logging
- Donation certificate/acknowledgment generation

3. Features to be Tested

Features mapped to project requirement IDs:

Requirement IDs	Test Focus	Priority
FR1–FR3	Registration, role-based login, profiles, password reset, OTP	Critical

FR4–FR7	Donor details, donation history, eligibility, next eligible date	High
FR8–FR11, FR30, FR33–FR34	Inventory CRUD, expiry, low-stock alerts, traceability, disposal, filtering/sorting	Critical
FR12–FR16, FR31, FR32, FR35	Availability search, requests, approval, compatibility, reservation, status and validation	Critical
FR17–FR19	Camp scheduling, donor registration, inventory update	Medium
FR20–FR21	Notifications and emergency broadcasts	High
FR22–FR23	Reports, statistics and dashboards	Medium
FR24–FR25	Donor and nearby blood-bank/hospital search	High
FR26–FR28	Admin management, approvals and audit logs	High
FR29	Post-donation certificate/acknowledgment	Low
NFR1–NFR2	Search response time and concurrent access	High
NFR3–NFR4, NFR9–NFR19, NFR23–NFR24	Scalability, reliability, usability, maintainability, interoperability, portability	Medium
NFR5–NFR8	Data protection, RBAC, secure communication and privacy	Critical
NFR20–NFR22	Duplicate allocation, transaction consistency and audit logging	Critical
SAFETY-1–SAFETY-4	Compatibility, expiry, duplicate allocation and recovery	Critical

4. Features Not to be Tested

- Internal implementation of third-party SMS/email providers beyond application integration behaviour.
- Availability and internal correctness of external hospital systems; only the BBMS integration interface will be tested where available.
- Underlying cloud-provider infrastructure or operating-system internals outside the application's control.
- Real patient treatment or clinical transfusion procedures; controlled fictitious data will be used.

5. Test Approach / Strategy

Levels:

- Unit tests (module/service level)
- Integration tests (frontend, backend, database, authentication, notifications and APIs)
- System tests (end-to-end BBMS workflows)
- Acceptance tests (major workflows against agreed criteria)

Types:

- Functional testing
- Regression testing
- Performance and load testing
- Usability testing
- Security testing

- Data integrity and transaction testing
- Compatibility testing

Test Design Techniques:

- Equivalence Partitioning
- Boundary Value Analysis
- Decision Table Testing for ABO/Rh compatibility
- State Transition Testing for blood-unit/request lifecycles
- Error Guessing / Negative Testing

Test Automation Strategy:

- API regression tests using Postman/Newman
- UI smoke tests for critical user journeys
- JMeter or equivalent performance/load tests
- Manual exploratory and usability testing

Entry Criteria: Stable build available and smoke-tested; test environment/database configured; test data/accounts prepared; selected features implemented and code-reviewed.

Exit Criteria: All planned test cases executed; no unresolved Critical defects; High defects resolved or formally dispositioned; performance, safety and security checks completed; Critical/High coverage at 100%.

Security Validation:

- Verify RBAC for donor, staff, hospital/patient and administrator roles.
- Verify OTP/authentication and secure password handling.
- Verify HTTPS/TLS communication.
- Verify protected data cannot be accessed by unauthorized users.
- Verify audit logging for critical actions.
- Test malicious/invalid inputs including SQL injection, XSS and malformed payloads.

6. Test Environment

Category	Planned Environment
Client	Desktop/mobile devices; latest two major versions of Chrome, Firefox, Edge and Safari.
Backend	Python/Django and/or JavaScript/Node.js on a staging instance.
Database	PostgreSQL or MySQL with controlled fictitious data.
API	REST APIs using JSON; secure token/session authentication.
Notifications	Test/stub SMS and email gateway in sandbox mode.
Performance	Controlled concurrent-user/load-testing environment.
Tools	Postman/Newman, browser developer tools, JMeter or equivalent, Git-based issue tracking.
Test Data	Fictitious donor, blood-unit, request, camp and transaction data; no real patient data.

7. Test Schedule

Milestones:

Phase	Milestone	Planned Timing
-------	-----------	----------------

Planning	Test plan review and sign-off	Week 1
Design	Test case design and RTM baseline	Weeks 1–2
Setup	Test environment and data setup	Week 2
Execution	Functional + integration testing	Weeks 3–4
Execution	Performance, security and usability testing	Week 4
Regression	Regression after defect fixes	Week 5
Acceptance	Acceptance testing	Week 5
Closure	Final test summary and sign-off	Week 6

8. Test Deliverables

- Software Test Plan
- Functional and non-functional test cases
- Automated/manual test scripts
- Test data sets and setup scripts
- Test execution records
- Defect reports
- Requirements traceability matrix
- Test summary report

9. Roles and Responsibilities

Role	Responsibility
QA/Test Lead (Both NFR AND FR)	Coordinate test planning, execution, defect triage and final status.
Test Engineers	Design/execute tests, record evidence and report defects.
Developers	Support debugging, fix defects and provide builds.
Project Team	Review requirements, test coverage and acceptance results.

10. Risks and Mitigation

Risk	Mitigation
Unstable build delays testing	Request incremental stable builds and smoke-test them.
Database/test-data inconsistency	Use controlled seed data and reset between cycles.
Notification gateway unavailable	Use mocks/stubs and separately verify integration.
Concurrent inventory requests cause inconsistent stock	Perform concurrency tests and verify atomic updates.
Sensitive data exposed during testing	Use fictitious data and restrict test-environment access.
Time constraints	Prioritize inventory, compatibility, fulfilment, authentication and security.

11. Assumptions & Dependencies

- Stable internet connection is available.
- Required test accounts and fictitious test data are available.
- Database/backend services are available during test windows.
- Notification test access or mocks/stubs are available.
- External hospital integrations, if tested, expose compatible APIs.

12. Suspension & Resumption Criteria

Suspend testing if:

- Test environment is unavailable/unstable enough to invalidate results.
- A blocking defect prevents substantial test execution.
- Required test data or backend services are unavailable.

Resume testing if:

- Blocking defects are resolved or an approved workaround is available.
- Environment is restored and smoke-tested.
- Required data and dependencies are available.

13. Defect Management

Severity:

Severity	Definition	Target SLA
S1 – Critical	Crash, data loss/corruption, or blood-safety rule violation.	Immediate / same day
S2 – High	Major feature broken with no workaround.	1 business day / 2 business days
S3 – Medium	Feature partially broken or workaround exists.	2 business days / 5 business days
S4 – Low	Cosmetic, wording or minor usability issue.	Best effort

Defect Lifecycle: New → Assigned → In Progress → Fixed → Retest → Closed. Failed retests reopen the defect; Deferred/Won't Fix requires QA-Lead + Project-Team sign-off.

Defect Reporting: Record defect ID, linked requirement/test-case ID, environment, build version, steps, expected/actual result, severity, priority, evidence and status.

14. Test Case Management & Traceability

The requirements traceability matrix maps project requirement IDs to test cases.

Requirement ID	Test Focus	Test Case ID(s)
FR8	Add/update/remove blood-unit records	TC-INV-01 to TC-INV-03
FR9	Expiry tracking and blocking expired units	TC-INV-04 to TC-INV-05
FR10	Stock visibility by blood group/location	TC-INV-06
FR11	Low-stock alert generation	TC-INV-07
FR16	Blood-group compatibility verification	TC-REQ-05
FR30	Blood-unit traceability	TC-INV-08
FR31	Temporary reservation of compatible units	TC-REQ-07
NFR1–NFR2	Search response time and concurrent access	TC-PERF-01 to TC-PERF-02
NFR5–NFR8	Data protection, RBAC, secure communication and privacy	TC-SEC-01 to TC-SEC-04
NFR20–NFR22	Duplicate allocation, transaction consistency and audit logging	TC-DATA-01, TC-DATA-02, TC-AUD-01

SAFETY-1–SAFETY-4	Compatibility, expiry, duplicate allocation and recovery	TC-SAF-01 to TC-SAF-04
-------------------	--	------------------------

15. Test Metrics & Reporting

Metrics collected:

Metric	Target
Planned test cases executed	100% before exit review
Pass rate	≥ 95%
Critical defects open at exit	0
High defects open at exit	0 unresolved / approved disposition only
Critical/High requirement coverage	100%
Medium/Low requirement coverage	≥ 90%
Search/availability response time	≤ 2–3 seconds at target load
Defect density	Tracked per module

Reports:

- Test execution status
- Defect summary
- Requirements coverage summary
- Final Test Summary Report

16. Approvals

Role	Name	Signature / Date
QA/Test Lead (NFR)	Shreenandan Jamadar	17/09/2026
Development Lead & Team Leader	Shreelakshmi G Bhat	17/09/2026
Project/Team Representative	Shreyash M Yadwad	17/09/2026
QA/Test Lead (FR)	Yesaswani Sangadala	17/09/2026

17. Appendix A — Sample Detailed Test Cases

TC-REQ-05 — Blood-group compatibility verification

Field	Detail
Requirement ID	FR16 / SAFETY-1
Priority	Critical
Preconditions	Hospital user logged in; multiple blood groups available.
Test Steps	1) Request O-negative. 2) Search compatible units. 3) Attempt to approve incompatible A-positive unit.
Expected Result	Only compatible units are returned; incompatible approval is rejected.

TC-INV-04 — Expiry tracking blocks expired units

Field	Detail
Requirement ID	FR9 / SAFETY-2
Priority	Critical
Preconditions	One unit expires today; one expired yesterday.
Test Steps	1) Query inventory. 2) Attempt to reserve expired unit.
Expected Result	Expiring-today unit is flagged; expired unit is excluded and reservation is rejected.

TC-DATA-01 — Prevent duplicate allocation under concurrent requests

Field	Detail
Requirement ID	NFR20 / SAFETY-3
Priority	Critical
Preconditions	Exactly one compatible unit is available.
Test Steps	1) Two hospital users request the same group concurrently. 2) Both attempt reservation.
Expected Result	Exactly one reservation succeeds; no duplicate allocation occurs.

18. Appendix B — Test Case Design Template

Field	Description
Test Case ID	Unique ID, e.g. TC-<MODULE>-<NN>
Title	Short description of what is verified
Requirement ID(s)	Linked FR/NFR/SAFETY IDs
Priority	Critical / High / Medium / Low
Preconditions	System/data state before execution
Test Steps	Numbered actions
Test Data	Specific input values
Expected Result	Observable outcome
Actual Result	Filled during execution
Status	Pass / Fail / Blocked / Not Executed